

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 3

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконав:

Студент групи 403-ТН

Солонецький Роман Миколайович

Перевірив:

Мізік Андрій Олегович

Полтава 2026

Тема: Вебдодаток для відстеження звичок (HabitTracker)

Для звіту використовую вже майже готовий проект до дипломної роботи.

Реалізація доступу до даних (Data Access Layer)

Для взаємодії з базою даних MongoDB у проекті використано підхід ODM (Object Data Modeling) за допомогою бібліотеки mongoose. Mongoose фактично виконує роль патерну Repository, абстрагуючи складність сирих запитів до БД. Створено чіткі схеми (Schemas) та моделі даних (User, Habit), які інкапсулюють логіку валідації та структуризації документів. Доступ до даних здійснюється через вбудовані методи моделей (наприклад, `.find()`, `.findById()`, `.save()`), що забезпечує безпеку та цілісність даних без необхідності написання прямих запитів базовою мовою СУБД.

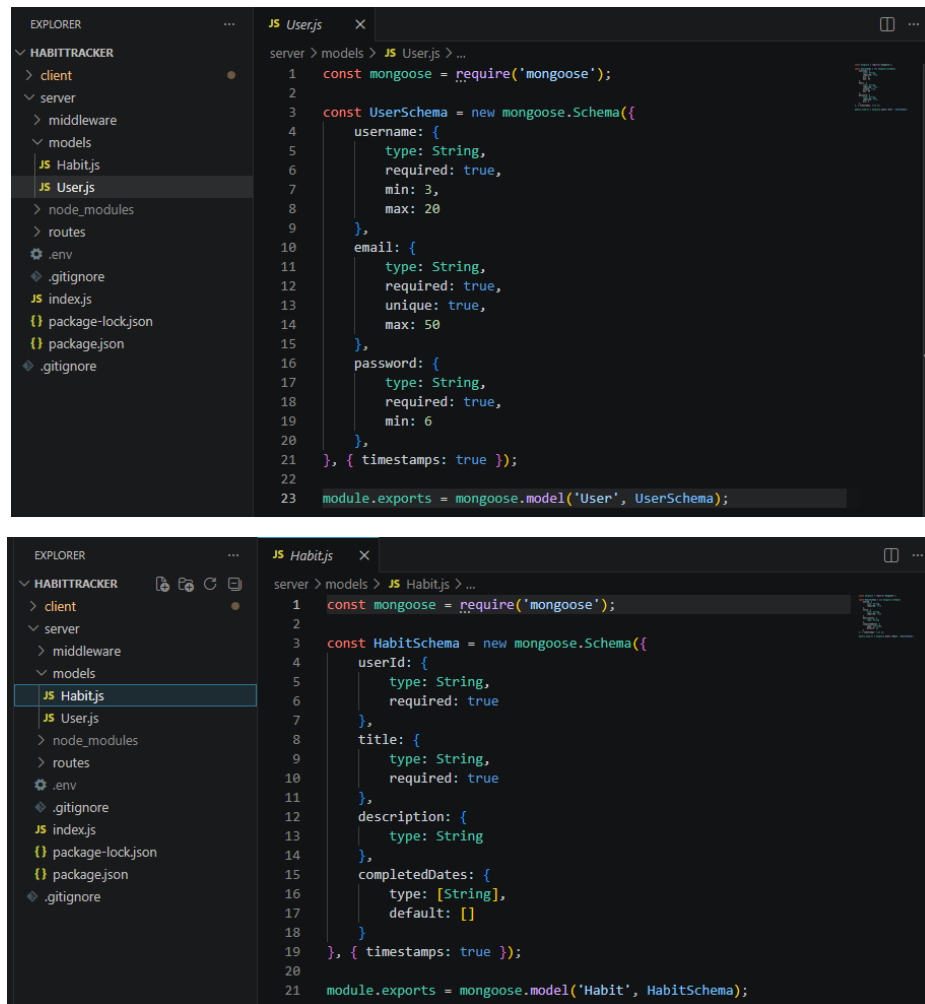


Рис 1-2 – Код моделей даних Habit та User

Архітектура API

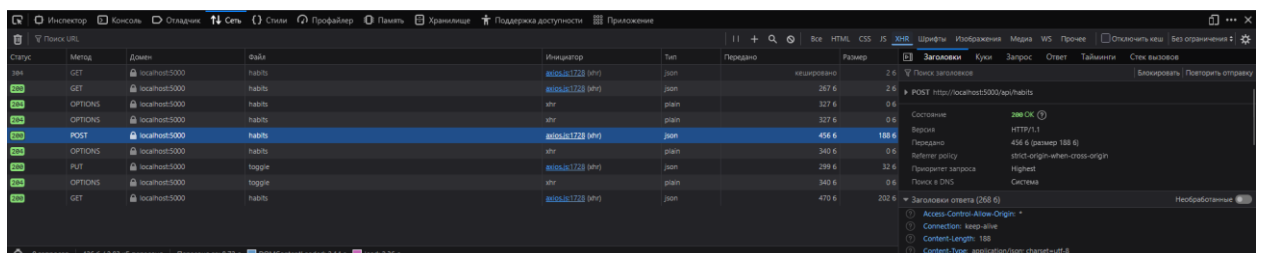
Програмний інтерфейс додатку (API) спроектовано згідно з архітектурним стилем REST (Representational State Transfer).

Використовуються семантично правильні HTTP-методи для операцій CRUD: GET (отримання списку звичок), POST (створення нової звички або запит до ШІ), PUT (оновлення статусу виконання звички), DELETE (видалення ресурсу).

```
29 router.put('/:id/toggle', verify, async (req, res) => {
30   try {
31     const habit = await Habit.findById(req.params.id);
32     const today = new Date().toISOString().split('T')[0];
33
34     if (habit.completedDates.includes(today)) {
35       await habit.updateOne({ $pull: { completedDates: today } });
36       res.json("Виконання скасовано");
37     } else {
38       await habit.updateOne({ $push: { completedDates: today } });
39       res.json("Звичку виконано!");
40     }
41   } catch (err) {
42     res.status(500).json(err);
43   }
44 });
45
46 router.delete('/:id', verify, async (req, res) => {
47   try {
48     await Habit.findByIdAndDelete(req.params.id);
49     res.json("Звичку видалено!");
50   } catch (err) {
51     res.status(500).json(err);
52   }
53 });
54
55 module.exports = router;
```

Рис 3 – Код операцій PUT, DELETE

Дотримано ієрархічної структури URI (наприклад, /api/habits/:id/toggle). Сервер повертає стандартизовані HTTP статус-коди залежно від результату операції (200 OK, 201 Created, 400 Bad Request для помилок валідації, 401 Unauthorized, 500 Internal Server Error). Взаємодія здійснюється у форматі JSON (формат application/json). Безпека ендпоінтів забезпечується через передачу JWT-токена у заголовках (headers).



Статус	Метод	Домен	Відомості	Ініціатор	Тип	Передано	Результат	Заголовки	Коди	Запит	Відповідь	Таймлайн	Список викликів
200	GET	localhost:5000	habits	axios:1722 (xhr)	json		200 OK		200	POST http://localhost:5000/api/habits	200 OK		Блокування / Поширення сторінки
200	OPTIONS	localhost:5000	habits	xhr	json	200 OK	200 OK		200		200 OK		
200	POST	localhost:5000	habits	axios:1722 (xhr)	json	456 B	200 OK	Version: HTTP/1.1 Content-Type: application/json; charset=utf-8	200		200 OK		
200	OPTIONS	localhost:5000	habits	xhr	json	200 OK	200 OK		200		200 OK		
200	PUT	localhost:5000	toggle	axios:1722 (xhr)	json	200 OK	200 OK	Referer policy: strict-origin-when-cross-origin Cache-Control: no-cache	200		200 OK		
200	OPTIONS	localhost:5000	toggle	xhr	json	200 OK	200 OK		200		200 OK		
200	GET	localhost:5000	habits	axios:1722 (xhr)	json	400 B	200 OK	Access-Control-Allow-Origin: *	200		200 OK		

Рис 4 – Повернення результатів

Архітектура (Services, чистота коду) та використання DTO/Mapping

Проект побудовано за трирівневою архітектурою (Контролери -> Сервіси -> Моделі), що забезпечує розділення відповідальності (Separation of Concerns). Бізнес-логіка винесена в окремий шар сервісів (HabitService), що дозволяє ізолювати обробку HTTP-запитів від безпосередньої логіки додатку (робота з ШД, підрахунок серій виконання).

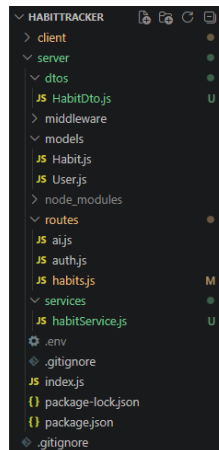


Рис 5 – Структура папок

Для передачі даних між сервером та клієнтом реалізовано патерн DTO (Data Transfer Object). Наприклад, UserDTO та HabitDTO використовуються для фільтрації чутливих даних (таких як хеші паролів або системні поля __v від MongoDB) перед їх відправкою на клієнтську частину, забезпечуючи безпечний мапінг об'єктів.

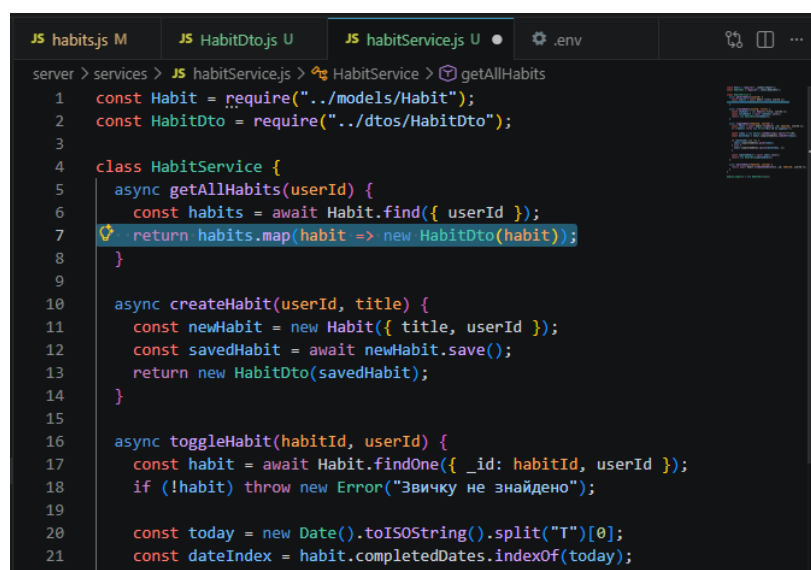


Рис 6 – Інтеграція DTO

Інтерфейс тестування API (Swagger)

Для документування та зручного ручного тестування API інтегровано інтерфейс Swagger UI (згідно зі специфікацією OpenAPI). Документація доступна за маршрутом /api-docs і містить опис усіх доступних ендпоінтів, необхідних параметрів, форматів тіла запиту та можливих HTTP-відповідей. Це дозволяє розробникам та тестувальникам взаємодіяти з API через зручний графічний інтерфейс, відправляти запити та перевіряти роботу бекенду (включаючи авторизацію через JWT) ізольовано від клієнтської частини.

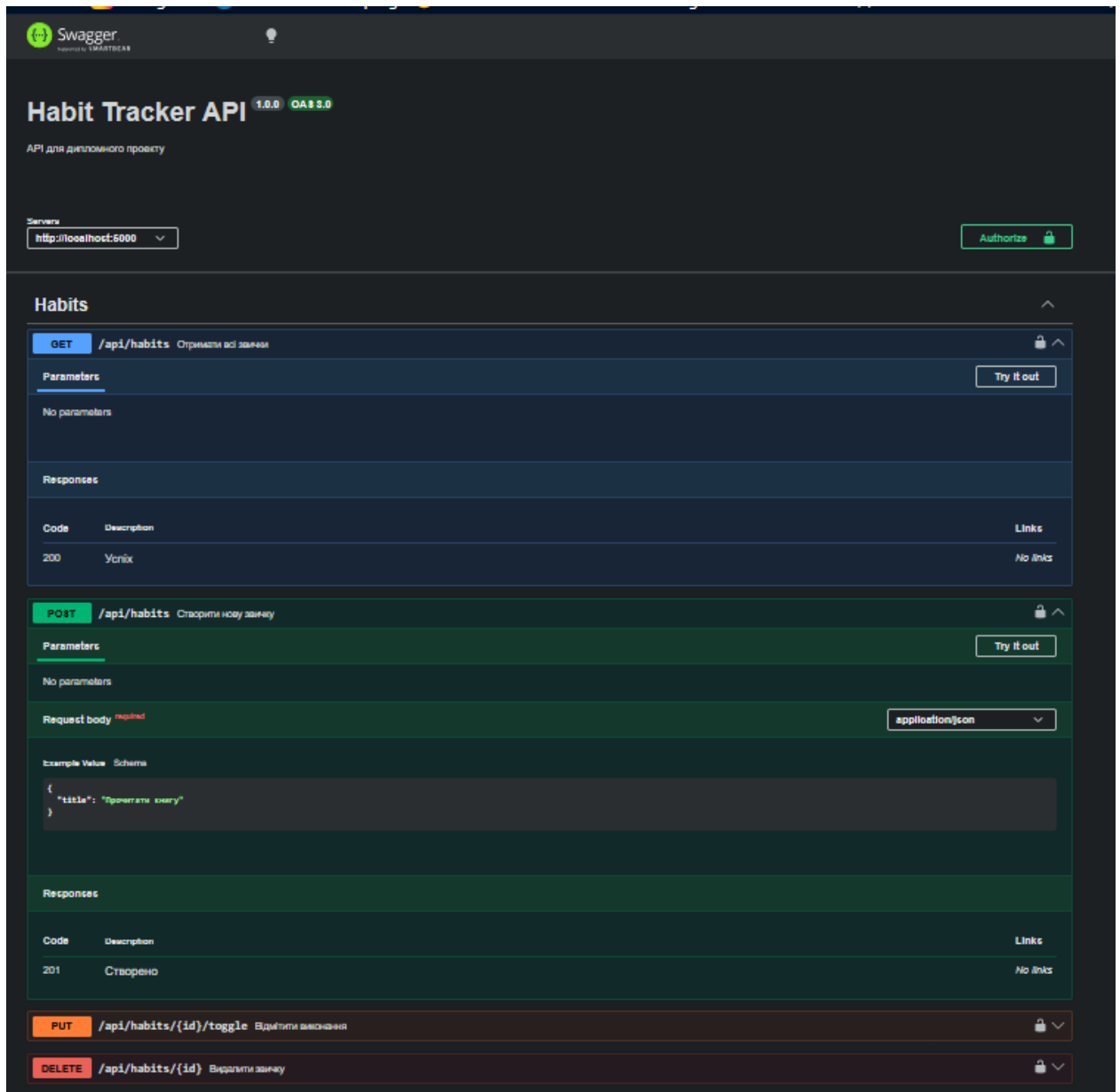


Рис 6 – Інтерактивна документація API реалізована за допомогою Swagger UI

Контрольні питання

1. Порівняйте REST, GraphQL та gRPC. У яких випадках доцільно використовувати кожен із них?

REST є стандартом для публічних веб-інтерфейсів, оскільки використовує зрозумілі HTTP-методи та URL-адреси для доступу до ресурсів. GraphQL доцільно обирати для мобільних додатків та складних фронтендів, де важливо отримувати багато пов'язаних даних одним запитом. gRPC найкраще підходить для внутрішньої комунікації між мікросервісами, оскільки забезпечує найвищу швидкість передачі даних у бінарному форматі.

2. Що таке проблема Over-fetching та Under-fetching даних? Як GraphQL вирішує ці проблеми порівняно з REST?

Over-fetching - це ситуація, коли сервер повертає зайві дані, які не потрібні клієнту, що марнує трафік. Under-fetching змушує клієнта робити кілька послідовних запитів, щоб зібрати всю необхідну інформацію. GraphQL вирішує ці проблеми, дозволяючи клієнту в одному запиті чітко вказати лише ті поля, які йому необхідні для відображення інтерфейсу.

3. Що таке Protocol Buffers (Protobuf)? Які його переваги перед JSON?

Protobuf - це механізм серіалізації даних у бінарний формат, що використовується в gRPC. Його головними перевагами перед текстовим JSON є значно менший розмір повідомлень, вища швидкість обробки процесором та суворі типізація даних на рівні схеми.

4. Як забезпечується версіонування API у REST та GraphQL?

У REST версіонування зазвичай реалізується через зміну шляху в URL (наприклад, /v1/ або /v2/), що чітко розділяє різні ітерації API. GraphQL використовує стратегію «безперервної еволюції» без версій: розробники просто додають нові поля до існуючої схеми, а застарілі позначають спеціальною директивою, зберігаючи підтримку старих клієнтів.

5. Поясніть концепцію Idempotency (ідемпотентності). Які методи/операції мають бути ідемпотентними?

Ідемпотентність означає, що повторне виконання одного і того ж запиту не змінить стан системи більше одного разу. Методи GET, PUT та DELETE мають бути ідемпотентними, щоб гарантувати безпеку при збоях мережі, тоді як POST не є ідемпотентним, оскільки кожен виклик створює новий об'єкт у базі даних.

6. Яка роль Schema Definition Language (SDL) у GraphQL або .proto файлів у gRPC?

Ці файли виступають у ролі суворого контракту між клієнтом та сервером, описуючи структуру доступних даних та методів. У GraphQL схема (SDL) дозволяє валідувати запити клієнта, а в gRPC .proto файл дозволяє автоматично генерувати код клієнта та сервера на різних мовах програмування.